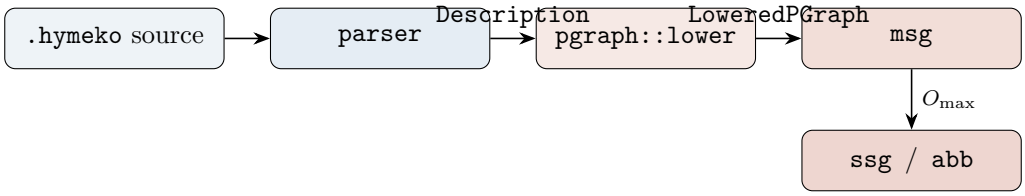
P-graphs in HyMeKo:
parse, lower, MSG, SSG, ABB

2026-05-05

This brief documents what landed: a P-graph encoding inside the canonical HyMeKo grammar (idiomatic — units are *hyperedges*), an AST → schema lowering, and Rust implementations of *Maximal Structure Generation* (MSG), *Solution Structure Generation* (SSG) and the *Accelerated Branch-and-Bound* (ABB) of Friedler, Tarján, Huang and Fan (1992). The pipeline is end-to-end on the worked HDA example.

The structural choice. In HyMeKo, an `@`-prefixed declaration is a hyperedge; without the `@` it is a hypernode. Both carry the same machinery (tags, an optional value, a body of `HyperItem`s) — the only structural difference is the role tag carried by `@`. This is exactly what a P-graph wants: a material is a passive resource (hypernode), an operating unit *is* a hyperedge connecting input materials to output materials. We thus encode units with `@` and put the I/O signature inside the edge body as a single signed hyperarc.

1 Pipeline



2 Encoding a P-graph in the HyMeKo grammar

A bipartite Material/Operating-Unit graph maps onto the canonical hypergraph grammar with three conventions:

- Materials are *hypernodes* tagged `<material>`.
- Operating units are *hyperedges* (`@`-prefix) tagged `<unit>`.
- Inside a unit body, place exactly one signed hyperarc whose signs are the unit's I/O signature: `-X` means the unit consumes X, `+X` means it produces X.

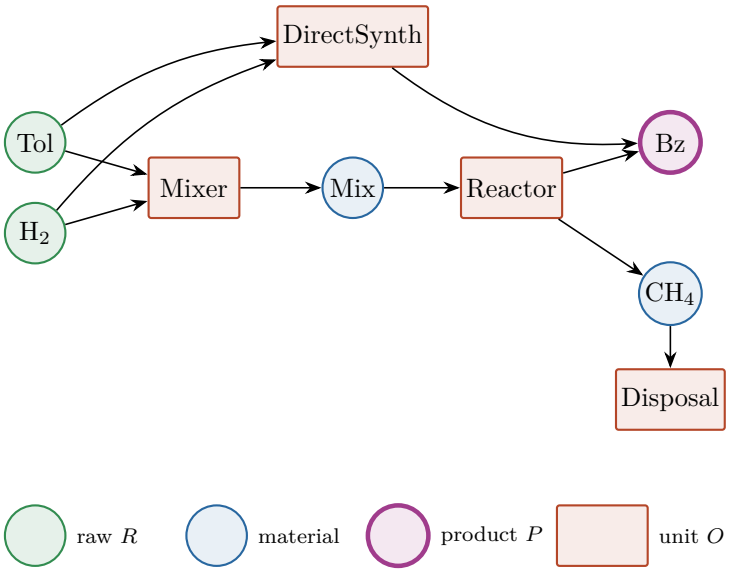
Sub-tags `<raw>` and `<product>` on materials populate the raw set R and the demand set P. The unit's scalar cost is the edge's numeric value (read by ABB); a `cost N;` child node is also accepted for back-compat.

```
HDA{}
context {
    Toluene <material, raw>;
    H2      <material, raw>;
    Mix     <material>;
    Benzene <material, product>;
    Methane <material>;
```

```
    @Mixer       <unit> 100 { (-Toluene, -H2, +Mix); }
    @Reactor     <unit> 250 { (-Mix, +Benzene, +Methane); }
    @DirectSynth <unit> 800 { (-Toluene, -H2, +Benzene); }
    @Disposal    <unit>  50 { (-Methane); }
}
```

Why this is the right encoding. The grammar's `NodeDecl` and `EdgeDecl` are structurally near-identical — both accept tags, an optional value, and a body of `HyperItem`s. The `@` prefix is purely a role marker, not a different kind of object. The lowering walks `HyperItem::Node` for materials and `HyperItem::Edge` for units and unifies them into the schema's (M, O, E_{dir}) triple. The signed-incidence semantics built into hyperarcs ($\pm$ for produce/consume) is then exactly what a P-graph needs.

The HDA P-graph



3 AST → `LoweredPGraph`

`hymeko_pgraph::lower` walks the parsed `Description`, descending one level into untagged-node wrappers like `context{...}`. Tagged `HyperItem::Node`s become materials; tagged `HyperItem::Edge`s become units. From each unit body, a single signed hyperarc is read and converted into one directed schema edge per signed reference (`-X` $\Rightarrow X \to U$, `+X` $\Rightarrow U \to X$). The bipartite invariant is enforced by `PGraphSchema::try_new`; a unit body using `~X` (neutral) is rejected. The lowered record carries every side-table the algorithms need:

$$\texttt{LoweredPGraph} = (\text{schema}, R, P, M, O, c, \text{in}, \text{out}),$$

where $c\colon O \to \mathbb{R}_{\geq 0}$ is the per-unit cost (default 1.0).

4 MSG — Maximal Structure Generation

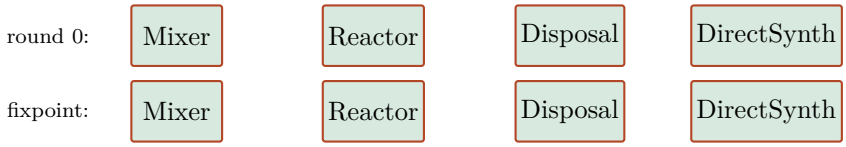
The maximal structure $O_{\max} \subseteq O$ is the largest subset that is both *forward*- and *backward*-feasible. The implementation iterates two trims to a fixpoint:

- **Forward.** Compute the producibility closure $C_R(O') = \text{lfp}\left[X \mapsto R \cup \{m \mid \exists u \in O', \text{in}(u) \subseteq X, m \in \text{out}(u)\}\right]$. Drop every $u \in O'$ whose $\text{in}(u) \not\subseteq C_R(O')$.

- **Backward.** Let useful $= P \cup \bigcup_{u \in O'} \text{in}(u)$. Drop every u whose $\text{out}(u) \not\subseteq$ useful.

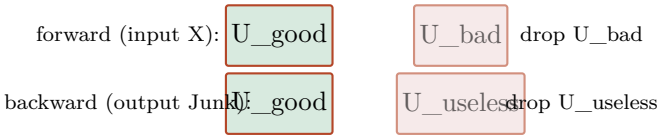
Iteration converges in $O(|O|)$ rounds. The backward rule is "every output is useful," *not* "some output reaches a product" — the latter wrongly drops legitimate consumer / disposal units (this was today's first MSG bug).

Walk on HDA



On HDA every unit is forward- and backward-feasible, so $O_{\max} = O$.

Walks that prune



5 SSG — Solution Structure Generation

SSG enumerates every $O' \subseteq O_{\max}$ that satisfies:

(a) *Input consistency:* for every $u \in O'$, $\text{in}(u) \subseteq C_R(O')$.

(b) *Demand:* every $p \in P$ lies in $C_R(O')$.

(c) *No-excess* (strict, default): every produced material that is not in $R \cup P$ is consumed by some $u \in O'$.

The implementation enumerates by bitmask over $O_{\max}$ (refuses silently for $|O_{\max}| > 30$, where ABB is the right tool).

On HDA with strict no-excess the feasible set is

$$\{\{\text{Mixer}, \text{Reactor}, \text{Disposal}\}, \{\text{DirectSynth}\}\}.$$

Relaxing (c) admits the by-product-leaving structure $\{\text{Mixer}, \text{Reactor}\}$ and a few more.

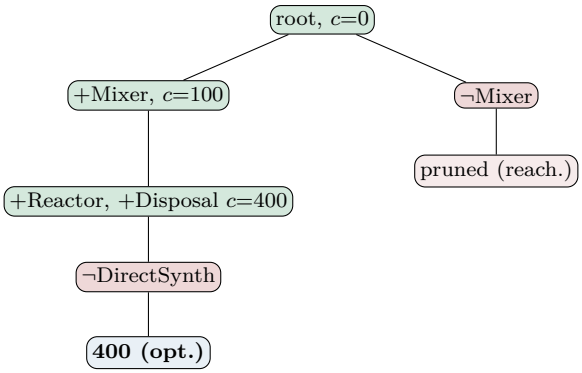
6 ABB — Accelerated Branch-and-Bound

ABB performs DFS over the include/exclude lattice of $O_{\max}$ with two pruning rules:

- **Inclusion bound.** Abandon a partial selection whose cost has already met or exceeded the current incumbent.
- **Reachability bound.** Let $\widehat{O} =$ included $\cup$ undecided. If $C_R(\widehat{O}) \not\supseteq P$, no descendant can be feasible — prune.

Branching order is fixed; each step takes "include" before "exclude", so a feasible incumbent appears early.

ABB tree on HDA (strict)



Excluding Mixer fails the reachability check (Mix is then unproducible), so half the tree dies at the second level. The right-most branch `+DirectSynth` is dominated by the inclusion bound at $c = 800 > 400$.

7 Empirical run on HDA

algorithm	output	cost
MSG	{Mixer, Reactor, Disposal, DirectSynth}	—
SSG (strict)	{{Mx,Rx,Ds}, {DS}}	—
SSG (relaxed)	4 structures incl. {Mx,Rx}	—
ABB (strict)	{Mixer, Reactor, Disposal}	400
ABB (relaxed)	{Mixer, Reactor}	350

The pipeline is exercised by `hymeko_pgraph/tests/pgraph_e2e.rs` (9 tests, all passing) on top of `hymeko_pgraph`'s 8 schema unit tests.

What's not done

A2/A3/A5 axiom checks (`axioms.rs`) are still parked — only A1 (product membership) and A4 (degree) fire today. SSG above 30 units, multi-optimum reporting in ABB, and reusing `hymeko_graph`'s cycle enumerator for recycle-loop-aware MSG remain follow-ups.